

组会汇报材料（PPT 大纲 + 逐页讲稿）

主题：双 Gate 强化学习的 Solver Selector (TSP 原型)

代码位置：my/（训练 gate，不训练 solver；调用 EasyNCO 平台 final/ 里的初始化器/迭代器）

关键日志与结果文件：

- 训练日志（示例）：`my/outputs/two_gate_tsp_seed2024_n100.log.txt`
- 训练曲线图：`my/outputs/training_analysis.png`（由 `my/analyze_log.py` 解析日志生成）
- 推理/求解输出（TSP100 uniform 10k）：
`my/outputs/test_tsp100_nums10000_uniform_solved_n100.txt`
- 推理/求解输出（TSP1000 uniform 128）：
`my/outputs/test_tsp1000_nums128_uniform_solved_n1000.txt`

建议汇报时长：10-15 分钟；主线是「论文方法 → 我认为的缺口 → 双 gate RL 方案 → 代码落地 → 现有结果与问题 → 下一步」。

Slide 1: 标题页（我做了什么）

PPT 内容（页内要点）

- 标题：双 Gate 强化学习的求解 pipeline 选择（TSP 原型）
- 一句话：把“选完整 solver”改成“选初始化 + 选迭代”，用 RL 解决组合爆炸的标注成本
- 我训练的是什么：只训练两个 gate (+ critic)，solver 来自 EasyNCO 预训练/实现

讲稿（口播）

大家好，我今天汇报的是我最近在做的一个方向：受到《Neural Solver Selection》那篇论文启发，我想把“实例级别选 solver”的思路进一步细化到“选择求解 pipeline 的两阶段组件”：第一阶段选初始化器，第二阶段在拿到初始化解以后再选迭代改进方法。

原论文是监督学习，需要为每个实例跑一遍所有 solver 得到离线标签；但如果我们把 pipeline 拆成初始化×迭代，组合数量会变成 $n \times m$ ，离线标注的成本爆炸，所以我现在用强化学习（Actor-Critic）直接用“最终解长度”做 reward，让 gate 自己试错学习。

今天主要讲三件事：方案怎么定义、代码框架怎么搭、现阶段跑出来的结果长什么样，以及目前暴露出的关键问题。

图/表建议

- 一张最简 pipeline 图：`data → gate1 → initializer → sol0 → gate2 → iterator → sol1`

Slide 2: 动机与背景（为什么要做 solver selection）

PPT 内容（页内要点）

- 前提：一个模型不可能适配所有实例（模型有偏好/偏置）
- 因此需要：实例级别的“选择/路由”（gating / routing）
- 论文背景：Neural Solver Selection（单 gate，solver zoo，监督学习）

讲稿（口播）

我先说一下最核心的前提：同一个神经求解器在不同实例上表现差异很大，本质上模型有偏好；如果我们只用一个 solver 去覆盖所有实例，一定会出现平均性能被“拖后腿”的情况。

所以一个自然的想法是做“选择”：给定一个实例，把它路由到更合适的 solver 或策略上。原论文就是做这件事：上层训练一个 selector，从一个固定 solver zoo 里为每个实例挑一个 solver，来降低平均 gap 并控制额外开销。

图/表建议

- 画一个“实例分布很杂 → 单一 solver 不稳定 → selector 分流”的示意图

Slide 3: 原论文方法回顾（单 gate + 监督学习）

PPT 内容（页内要点）

- 结构：instance → feature/encoder → selection model → solver scores
- 监督信号：离线标签 `raw_label.pkl`（每实例每 solver 的 cost/time/gap）
- 评估：离线模拟 top-1 / top-k / rejection / top-p 的 gap/time

讲稿（口播）

原论文的关键点是：他们不训练 solver，本仓库也默认不包含 Omni、DIFUSCO、LEHD 等 solver 的训练；相反，他们把每个 solver 在每个实例上的求解结果离线固化成标签文件。训练时学习的是一个“选择器”：输入实例特征，输出对每个 solver 的分数（或者排序），然后用这个分数做离线的策略模拟，比如 top-k、rejection 等。

这套方法的优势是训练稳定、计算代价可控（因为标签固定）；但它依赖一个很强的前提：你只需要在一个“固定 solver 集合”上做选择，而且标注成本（对每个实例跑所有 solver）可接受。

图/表建议

- 画出论文的三段式：Feature extraction → Selection model → Selection strategies（top-k/rejection/top-p）

Slide 4: 我认为是的缺口（pipeline 假设不成立 + 组合爆炸）

PPT 内容（页内要点）

- 论文隐含假设：一个 solver 的 pipeline 适用于所有数据
- 但 pipeline 可拆：初始化（construction）+ 迭代改进（improvement）
- 两阶段对实例的“偏好”可能不同 → 需要分别选择
- 拆开后组合： $n(\text{init}) \times m(\text{iter})$ ，监督标注成本爆炸 → 改用 RL

讲稿（口播）

我觉得原论文里有一个隐含假设：每个 solver 都是一个“整体”，你只要在整体之间选择就够了。但在很多 NCO 方法里，pipeline 可以很自然地拆成两段：先产生一个初始解（初始化/构造），再用迭代搜索去改进（2-opt、RRC、大邻域搜索等）。

这两段对实例的偏好可能完全不同：比如某个初始化器在某类实例上很快给出不错的初始解，但它的改进策略不一定适合；反过来，一个改进器可能更适合从某种结构的初始解出发。

如果我们真的想选“pipeline”，那就不是选 n 个 solver，而是选 n 个初始化器和 m 个迭代器的组合，总组合 $n \times m$ 。监督学习的离线标注需要把每个实例的所有组合都跑一遍，这基本不可行，所以我转向强化学习，让 gate 用最终解长度当 reward 自己探索组合。

图/表建议

- 用一个二维网格画 “initializer × iterator = 组合空间”，标注监督学习需要全网格打标签

Slide 5: 方案定义（双 gate：先选初始化，再选迭代）

PPT 内容（页内要点）

- Gate1: 输入 `data`，动作是选初始化器 $a1 \in \{\text{init}_1.. \text{init}_n\}$ ，得到 `so10`
- Gate2: 输入 `data + so10`，动作是选迭代器 $a2 \in \{\text{iter}_1.. \text{iter}_m\}$ （固定迭代步数），得到 `so11`
- 不考虑时间：先固定迭代次数，只学习“选哪种迭代方式”

讲稿（口播）

方案很直白：我把选择过程变成一个两步决策。第一步 Gate1 只看原始实例数据，选择一个初始化器，跑出一个初始 tour `so10`。第二步 Gate2 在看到 `so10` 以后再选一个迭代器，对 `so10` 做固定步数的改进，得到最终 tour `so11`。这里我刻意简化：暂时不把时间或步数当决策变量，而是把“迭代次数”当常量，比如 RRC 固定 10 步，2-opt 固定 200 次尝试——这样 Gate2 学到的是“哪一种改进方式更适合这个实例+初始解”，先把框架跑通。

图/表建议

- pipeline 图（建议在这一页放“最终版”流程图）

Slide 6: 强化学习定义（AC, reward=-length）

PPT 内容（页内要点）

- Reward: `reward = -length(so11)`（越短越好）
- Actor: 两个 gate（两个分类分布），分别输出 $\pi_1(a1|s0)$ 、 $\pi_2(a2|s1)$
- Critic: 预测 baseline（降低方差）
- 关键：loss 里同时包含两次决策（Gate1 + Gate2）

讲稿（口播）

强化学习的定义我用最小化长度的标准写法：reward 直接取负的 tour 长度 `reward=-length`。这样最大化 reward 等价于最小化长度。

策略有两部分：Gate1 输出一个离散分布采样初始化器；Gate2 输出另一个离散分布采样迭代器。最终 reward 是跑完“初始化+迭代”之后才得到的，所以同一个 reward 要同时归因给两次动作。

我使用 Actor-Critic：actor 用 REINFORCE 形式更新，但用 critic 作为 baseline 去减小方差。并且为了更合理地做 credit assignment，我用两个 critic：一个看 Gate1 的状态特征 `s0`，一个看 Gate2 的条件特征 `s1`（即 `data+so10` 的特征），分别作为两次决策的 baseline。

图/表建议

- 一行公式（只放最核心的）：
$$L_{\text{actor}} = -E[\log \pi_1(a1|s0) \cdot A1 + \log \pi_2(a2|s1) \cdot A2]$$
$$A1 = r - b1(s0), A2 = r - b2(s1)$$

Slide 7: 网络结构 (特征提取 + Gate MLP + Critic MLP)

PPT 内容 (页内要点)

- Feature extraction: 复刻论文编码器 (Transformer 风格全连接自注意力; 可选层次化 pooling)
- Gate1: `feat1(data) → MLP → logits over initializers`
- Gate2: `feat2(data, sol0) → MLP → logits over iterators`
 - `feat2 = feat1 + len0 + tour` 简单统计量 (让 Gate2 真正“看到 sol0”)
- Critic: 两个 value MLP, 分别预测 `v1(feat1)` 和 `v2(feat2)`

讲稿 (口播)

为了让整个框架尽量和论文对齐, 我在 `my/` 里重新实现了一份“论文同款风格”的实例编码器: 本质是一个 Transformer 风格的全连接注意力 encoder, 对点集做自注意力, 然后做 pooling 得到一个实例级 embedding。对于 TSP, 这相当于一个 set encoder / graph attention encoder。

在这个 embedding 基础上, Gate1 和 Gate2 都是简单的 MLP 分类器: 输出 logits, softmax 后形成离散分布。关键在于 Gate2: 它不能只看原始实例, 否则就退化“两个 gate 都在做同一件事”。所以我在 Gate2 的特征里显式加入了 `sol0` 的信息: 例如 `len0` (初始解长度) 以及一些 `tour` 的统计量, 这样 Gate2 的决策是“在这个初始解条件下选哪种迭代更划算”。

Critic 也是 MLP: `value1(feat1)` 给 Gate1 用, `value2(feat2)` 给 Gate2 用。

图/表建议

- 画三块: Encoder → (Gate1 MLP, Gate2 MLP) + (Value1 MLP, Value2 MLP)
- 标注 Gate2 输入比 Gate1 多了 `sol0` 的统计特征

Slide 8: 代码落地 (如何“不改 final, 只调用 final”)

PPT 内容 (页内要点)

- 目标: 不改 `final/`; 所有新实现都放 `my/`
- 做法:
 - `my/easynco_bootstrap.py`: 运行时把 `EasyNCO.*` 映射到 `final/` (解决绝对导入)
 - `my/solver_zoo.py`: 把不同 solver 的接口包装成统一的 `solve()/improve()`
 - `my/train_two_gate_tsp.py`: 两步采样 + 求解 + AC 更新

讲稿 (口播)

实现上我尽量遵守一个约束: `final/` 是我自己的平台项目, 我不想为了这个原型去改动平台代码, 所以所有 glue code 都写在 `my/` 里。

最大的工程问题其实是“导入路径”: `EasyNCO` 里很多代码用 `EasyNCO.xxx` 的绝对导入, 但在这个仓库里它实际位于 `final/`。所以我写了 `my/easynco_bootstrap.py` 在运行时给 Python 注入模块别名, 把 `EasyNCO` 指向 `final`, 这样平台代码可以原封不动跑起来。

另外, 各个方法的 initialization/iteration 接口并不统一: 有的返回的是 `TensorDict`, 有的是 `tour permutation`, 有的是 `successor` 表示。所以我在 `my/solver_zoo.py` 里写了薄包装, 把它们统一成 `solve(coords)→`

`(tour, length)` 和 `improve(coords, sol0)→(tour, length)`, 这样训练循环只关心“动作 id 对应哪个模块”和“最终长度是多少”。

图/表建议

- 放 3 个文件路径作为“系统结构图”：`my/train_two_gate_tsp.py` / `my/solver_zoo.py` / `my/easynco_bootstrap.py`

Slide 9: 训练/评估设置（我到底怎么跑的）

PPT 内容（页内要点）

- 训练数据：实时生成无限数据（对齐论文合成分布）
 - 支持：uniform + Gaussian mixture（按实例随机选分布）
- 评估数据：固定 eval set（由 `eval_seed` 一次生成，可复现）
- Solver zoo（示例）：`init={LEHD, ELG, DIFUSCO}`；`iter={None, RRC(LEHD), 2-opt}`
- baseline：本次展示使用 `baseline=critic`

讲稿（口播）

我这版框架训练数据不是从磁盘读固定训练集，而是每一步都在线生成一批新实例，做到“无限数据”，避免过拟合到某个固定训练集；评估时则生成一个固定 eval set，保证可复现。

分布上，我完全按论文的合成数据方式来写：uniform 以及 Gaussian mixture（采样若干个高斯模态，然后用 min-max scaler 映射到 [0,1]），并且训练时可以把多个分布混在一起，每个实例随机抽一个分布。

Solver zoo 这边，我先把框架跑通：初始化和迭代器来自 `final/`。这次展示我主要用 LEHD、ELG、DIFUSCO 做初始化器，迭代端用“什么都不做”、LEHD 风格的 RRC、以及一个可控的 2-opt。

图/表建议

- 放一行命令（训练命令）：

```
python my/train_two_gate_tsp.py --problem_size 100 --batch_size 64 --train_steps 20000 --train_dists gaussian uniform --eval_dist gaussian --init_zoo lehd elg difusco --iter_zoo none rrc_lehd two_opt --rrc_steps 10 --baseline critic --device cuda:0 --log_path my/outputs/two_gate_tsp_seed2024_n100.log.txt
```

Slide 10: 训练曲线与现象（gate 的“塌缩 → 跳变”）

PPT 内容（页内要点）

- 关键现象（来自 `my/outputs/two_gate_tsp_seed2024_n100.log.txt`）：
 - 前期 Gate2 很快塌缩到 `none`（不迭代）
 - 但在 ~step 740 之后 Gate2 突然切换到 `rrc_lehd_10`，eval 立刻变好
 - entropy 下降到接近 0（探索不足）
- 关键数字（eval on Gaussian dist）：
 - `[eval step 700] mean_len=5.7592`（Gate2=none）
 - `[eval step 800] mean_len=5.6610`（Gate2=rrc_lehd_10）
 - 相对改善约 $(5.7592 - 5.6610) / 5.7592 \approx 1.7\%$

讲稿（口播）

这一页是我想重点展示的训练现象：框架已经“能学东西”，但也暴露了典型的 RL 问题。

在训练早期，Gate1 很快就偏向某个初始化器（这里几乎全选 LEHD），Gate2 更明显：迅速塌缩为永远选 `none`，也就是“初始化完就结束”。这其实很好理解：迭代通常更慢、更容易引入噪声；如果一开始策略随机，RRC/2-opt 不一定立刻带来稳定收益，梯度噪声很大，于是策略就倾向于选最保守的动作。

但有意思的是，在 step 740 左右 Gate2 出现了一个“跳变”：从 `none` 切到 `rrc_lehd_10`，并且 eval 的 mean length 立刻从 5.7592 降到 5.6610，改善大概 1.7%。这说明：一旦 Gate2 真正探索到“RRC 在这个分布上有稳定收益”，它就会迅速把概率推到 1。

同时，这也提示我下一步要解决的问题：entropy 几乎掉到 0，探索非常不足，策略很容易早早陷入局部最优（比如一直选 none）。

图/表建议

- 直接贴 `my/outputs/training_analysis.png`
- 在图上用箭头标出 step~740~800 的“Gate2 切换点”
- 补一个小表格（建议放 PPT 右侧）：
 - step 700: Gate2=none:1.00, mean_len=5.7592
 - step 800: Gate2=rrc_lehd_10:1.00, mean_len=5.6610

Slide 11: 在真实测试集上跑 (TSP100 uniform 10k)

PPT 内容（页内要点）

- 数据: `final/data/datasets/test_dataset_tsp_uniform/test_tsp100_nums10000_uniform.pt`
- 推理输出: `my/outputs/test_tsp100_nums10000_uniform_solved_n100.txt`
- 结果摘要：
 - `mean_len=7.790962 best_len=6.689018`
 - Gate1 恒选 `lehd`；Gate2 恒选 `rrc_lehd_3`
- 注意：推理时 `rrc_steps=3`，而训练日志示例里 `rrc_steps=10`（配置不一致会影响数值）

讲稿（口播）

为了确认“我确实在调用 EasyNCO 求解器，而不是在做假计算”，我把训练好的 gate 拿到平台自带的测试集上跑了一下。这个测试集是 TSP100、10000 个 uniform 实例。

当前模型在这个测试集上的 mean length 大约是 7.79，best 是 6.69。与此同时，gate 的行为非常确定：Gate1 全部选 LEHD 初始化，Gate2 全部选 RRC（但这里是 `rrc_steps=3`）。

这里我特别强调一个 caveat：我训练日志里展示的那次 run 用的是 `rrc_steps=10`，但我跑这个推理时为了省时间默认用了 `rrc_steps=3`，所以结果不应直接和训练时的 eval 数值做横向比较；更合理的做法是把推理配置和训练配置对齐，再做“同分布/同设置”的对比。

图/表建议

- 放两行关键 summary（直接截图/复制文件末尾的三行）
 - 如需更直观：放 gate 分布条形图（lehd 100%，rrc 100%）
-

Slide 12: 跨规模测试 (TSP1000 uniform 128)

PPT 内容 (页内要点)

- 数据: `final/data/datasets/test_dataset_tsp_uniform/test_tsp1000_nums128_uniform.pt`
- 推理输出: `my/outputs/test_tsp1000_nums128_uniform_solved_n1000.txt`
- 结果摘要:
 - `mean_len=25.812925 best_len=25.220968`
 - Gate1 恒选 `elg`; Gate2 恒选 `none`
- 解读: problem size OOD 时, gate 的偏好会发生变化 (说明它确实在“根据特征路由”)

讲稿 (口播)

我还做了一个非常粗的 OOD 测试: 把同一个 gate (训练时 `problem_size=100`) 直接拿去跑 TSP1000。

结果上, 均值长度在 25.81 左右 (这主要是因为 N 变大, 长度自然会变大), 但更重要的是 gate 的选择发生了变化: Gate1 变成总选 ELG, Gate2 则总选 none, 不再跑 RRC。

这件事我现在的解读是: 框架在“行为层面”是成立的——它确实会根据输入特征 (包括规模变化引起的 embedding/统计差异) 做不同决策; 但从性能角度, 这个 OOD 结果是否合理还需要更严谨的对比 (例如固定用某个 solver 的长度作为 baseline 对照), 以及需要确保推理配置、迭代预算等都一致。

图/表建议

- 放一张小表: `N=100` → 选 `lehd+rrc`; `N=1000` → 选 `elg+none`
- 强调“选择变化”而不是强调绝对长度

Slide 13: 我从结果里学到的东西 (问题列表)

PPT 内容 (页内要点)

- 典型 RL 问题: 策略塌缩、探索不足 (`entropy`→0)
- 训练/推理配置容易不一致 (例如 `rrc_steps`), 影响可比性
- 计算代价: 每一步都要真实求解 (不像论文离线标签), 训练更慢
- credit assignment: Gate2 的回报更直接, Gate1 更“远因”→ 需要更好的 baseline/正则

讲稿 (口播)

目前结果给我的最大收获不是“已经很强”, 而是把问题暴露得很清楚:

第一, 策略塌缩非常明显, Gate2 早期总选 none, 这会让训练卡在局部最优; 虽然它后来也能跳到 RRC, 但依赖偶然探索。后续我需要加入更系统的探索机制, 比如 `entropy bonus`、温度、`epsilon-greedy`、或者在 zoo 层面做 curriculum。

第二, 训练和推理的配置必须严格对齐, 否则数值对比没有意义。最典型的就是 `rrc_steps`, 我已经看到它会直接改变最终长度。

第三, 和论文的监督学习完全不同: 我的 RL 每一步都要“真正跑一次求解器”, 因为 reward 来自真实解长度, 不可能像离线标签那样直接查表。这意味着训练速度会慢很多, 所以工程上要更关注批处理、缓存、以及 zoo 的选择规模。

第四, 从 credit assignment 角度, Gate2 的状态更接近 reward, 所以更好学; Gate1 更远, 可能需要更强的 baseline 或者更结构化的奖励分解。

图/表建议

- 画一个“训练成本对比”示意: 监督学习查表 vs RL 真实求解

Slide 14: 下一步计划（我接下来准备怎么推进）

PPT 内容（页内要点）

1. **对齐实验设置**：训练/推理统一 `rrc_steps`、`iter budget`；建立一组可复现的对照表（固定 solver vs gate）
2. **改进探索**：加 entropy bonus 或温度退火；避免早期塌缩
3. **扩大/清洗 iter zoo**：优先加入“本质不同”的迭代（RRC、2-opt、神经 2-opt），避免重复的 2-opt 变体
4. **更严格评测**：
 - 同分布（uniform / gaussian）内测
 - OOD（规模、分布、TSPLIB）外测
5. **未来（暂不做）**：引入 time-aware reward、可迭代次数（把时间也当动作）

讲稿（口播）

最后说一下接下来我会怎么做。短期目标是把“结果可对比、可复现”这件事先做扎实：

我会把训练和推理的 solver 参数完全对齐，然后做几条非常清楚的 baseline：比如固定用 LEHD 初始化 + none、固定用 LEHD + RRC、固定用 ELG + none，然后看 gate 在相同预算下能不能学到“按实例选更好的组合”。

与此同时，我会优先解决探索：现在 entropy 很快掉到 0，后面很多改进都没意义，因为策略不探索。加 entropy bonus/温度退火是最直接的办法。

中期我会扩展 iter zoo，但不会盲目加“看起来很多其实都是 2-opt”的重复方法，而是优先把本质不同的迭代机制纳入，保证动作空间的多样性。

长期再考虑更真实的目标：把时间/迭代次数也纳入决策，做 multi-objective 或 constrained RL，但这一步我暂时不做，先把核心框架与评测闭环打通。

图/表建议

- 用 checklist 形式列 5 条 next steps

Slide 15（可选备份）：如何复现实验（命令与文件）

PPT 内容（页内要点）

- 训练：
 - 需要把预训练 solver 权重放在 `model/`（例如 `model/lehd_tsp100.ckpt` 等）
 - 示例命令见 Slide 9
- 求解/输出 txt：
 - `python my/solve_dataset_tsp.py --gate_ckpt ... --data_path ... --out_path ... --device cuda:0 --batch_size 128 --model_dir model --rrc_steps 10`
- 日志与图：
 - 训练会写 `--log_path` (txt)
 - `python my/analyze_log.py my/outputs/xxx.log.txt -o my/outputs/xxx.png` 生成曲线图 (png)

讲稿（口播）

这页主要是备份，方便会后同学复现或者我自己下次跑的时候不忘参数。核心要点是：我训练 gate 需要依赖 EasyNCO 的求解器实现和对应权重文件，所以 `model/` 里要有对应方法的 `ckpt`。训练脚本会把配置打印到日志开头，后续分析脚本直接从日志里解析曲线。

我准备的“结果解读”一句话总结（放在最后一页/口头收尾）

- 1. 框架闭环已经跑通：**Gate1/2 的动作会真实调用 EasyNCO 的初始化与迭代**，reward 来自真实 tour length。
- 2. 在合成分布（Gaussian）上，策略经历“塌缩→跳变”，最终学会在该分布下稳定选择 `LEHD + RRC`，eval mean length 从 5.7592 改善到 5.6610（约 1.7%）。
- 3. 在外部 uniform 测试集上也能稳定工作（输出 tour 到 txt），但数值对比需要严格对齐迭代预算（如 `rrc_steps`）。
- 4. 当前主要瓶颈是：**探索不足/策略塌缩** 与 **实验设置不一致导致的不可比**；下一步优先做探索正则 + 统一评测 protocol。

各指标计算方式说明

核心指标速查表

指标	计算代码	含义
adv (adv_mean)	<code>adv2.mean().item()</code>	Gate2优势函数的均值
adv_std	<code>adv2.std(unbiased=False).item()</code>	Gate2优势函数的标准差
critic_mse	<code>critic_loss.item()</code>	Critic的MSE损失
entropy	<code>(dist1.entropy().mean() + dist2.entropy().mean()).item()</code>	两个Gate的熵之和
loss	<code>loss.item()</code>	总损失 (<code>actor_loss + critic_coef × critic_loss</code>)

计算

1. 优势函数计算

```
adv1 = reward - baseline1 # Gate1优势
adv2 = reward - baseline2 # Gate2优势
```

2. Actor损失

```
actor_loss = -(logp1 * adv1.detach() + logp2 * adv2.detach()).mean()
```

- `logp1/logp2`: 两个Gate对应动作的对数概率
- `detach()`: 阻断梯度回传, 避免优势函数的梯度影响Actor网络更新
- 负号: 将最大化收益转换为最小化损失的优化目标

3. Critic损失

```
# 以真实奖励为基准, 计算预测值与真实值的MSE
critic_loss1 = ((reward - value1_pred) ** 2).mean()
critic_loss2 = ((reward - value2_pred) ** 2).mean()
critic_loss = 0.5 * (critic_loss1 + critic_loss2)
```

4. 总损失

```
loss = actor_loss + cfg.critic_coef * critic_loss
```

- `cfg.critic_coef`: Critic损失的权重系数, 用于平衡Actor和Critic的学习速率

5. 熵

```
entropy = dist1.entropy().mean() + dist2.entropy().mean()
```

熵值越高表示策略的随机性越强 (多样性越好), 越低表示策略越确定, 用于避免策略过早收敛到局部最优。